



CAREERSENSE ACADEMY

BEGINNER LEARNING HANDBOOK

APIs for Beginners

Requests, responses, REST, JSON, authentication, errors, testing, and your
first API integration

**For students • career switchers • analysts • junior developers • product professionals • AI
builders**

CareerSense Learning Series • careersenseai.com/resources

How to Use This Handbook

APIs are the invisible connectors behind modern software. When a weather app loads a forecast, a payment page confirms a transaction, or an AI assistant calls a tool, an API is often carrying the request and returning the result. This handbook teaches that communication from first principles.

The goal is not to memorize terminology. You will learn to read an API request like a sentence: where it is going, what action it asks for, what data it carries, how access is controlled, and how to interpret the response.

What each chapter contains

Each chapter follows a steady learning rhythm: a short introduction, learning objectives, plain-English concepts, worked HTTP and JSON examples, common mistakes, best practices, revision notes, practice questions, and hands-on exercises.

BEGINNER PROMISE

Examples stay deliberately small. You do not need a backend framework to understand this book. A browser, curl, Postman-like client, or a few lines of Python are enough to practise the core ideas.

Recommended practice setup

- Use a public demo API or a local mock API for practice.
- Keep a text file with requests that worked and errors you learned from.
- Inspect status code, headers, and body separately.
- Never paste real secrets into screenshots, repositories, or learning notes.

CONTENTS

What You Will Learn

Chapter	Topic
01	What an API Is and Why It Exists
02	HTTP: URLs, Methods, Headers, and Bodies
03	REST and Resource-Oriented Design
04	JSON and Data Contracts
05	Status Codes, Errors, and Debugging
06	Authentication and API Security Basics
07	Testing APIs with curl, Postman, and Python
08	CareerSense Mini Project: Build a Job Search API Client

Closing the book: API cheat sheet • glossary • professional habits • 20-question quiz • answer key.

01

What an API Is and Why It Exists

An API is a contract for communication between software systems. One side asks for something in a defined format; the other side returns a defined result.

LEARNING OBJECTIVES

- Define API in plain English
- Identify client and server roles
- Explain request and response
- Recognize common API use cases
- Distinguish an API from a user interface
- Read a simple endpoint

The simplest mental model

Imagine a restaurant. You are the client, the kitchen is the server, and the menu is the API contract. You do not walk into the kitchen and tell the chef how to cook. You choose an allowed request from the menu, provide the required details, and receive a result.

CORE IDEA

An API defines what another system is allowed to ask for, how to ask, and what kind of answer to expect.

Request and response

Part	Meaning	Example
Client	System making the request	CareerSense web app
Server	System receiving and processing it	CareerSense API
Request	The ask	Get profile 42
Response	The answer	Profile data or an error

--	--	--

HTTP REQUEST

GET https://api.example.com/users/42

RESULT

200 OK

```
{"id":42,"name":"Aarav"}
```

Where APIs show up

- Mobile apps talking to backend servers.
- Websites loading profile, order, or analytics data.
- Payment systems connecting merchants to payment providers.
- AI agents calling search, calendar, CRM, or database tools.
- Internal company services exchanging data.

COMMON MISTAKE

Thinking an API is only a URL. The URL matters, but the complete request can also include a method, headers, query parameters, authentication, and a body.

QUICK REVISION NOTES

- API = application programming interface.
- Client asks; server responds.
- The API contract defines allowed communication.
- Endpoints expose capabilities or resources.
- APIs are used by web, mobile, data, cloud, and AI systems.

Practice Questions

- Q1** What is an API in your own words?
- Q2** What is the difference between a client and a server?
- Q3** What are the two halves of an API interaction?
- Q4** Why is the restaurant analogy useful?

Hands-on Exercises

- 1 List three APIs you probably use indirectly every day.
- 2 Write a plain-English request and response for a movie-search API.

02

HTTP: URLs, Methods, Headers, and Bodies

Most web APIs communicate over HTTP. Understanding a request means understanding its destination, action, metadata, optional parameters, and payload.

LEARNING OBJECTIVES

- Break a URL into useful parts
- Explain GET, POST, PUT, PATCH, and DELETE
- Understand headers
- Use query parameters
- Recognize request bodies
- Compare safe and unsafe operations

Anatomy of a URL

URL

`https://api.careersense.example/v1/jobs?status=applied&limit=20`

Piece	Example	Purpose
Scheme	https	Secure HTTP
Host	api.careersense.example	Server address
Path	/v1/jobs	Resource location
Query	status=applied&limit=20	Filters or options

HTTP methods

Method	Typical intent	Example
GET	Read	Get a job application

POST	Create	Create a new application
PUT	Replace	Replace a full resource
PATCH	Update part	Change status only
DELETE	Remove	Delete a draft record

HTTP REQUEST

POST /v1/applications

Content-Type: application/json

```
{"company":"Acme","role":"Data Analyst"}
```

BEST PRACTICE

Choose the method that matches the intent. A GET request should not secretly create or delete data. Predictable methods make APIs easier to understand and safer to operate.

Headers are metadata

- Content-Type tells the server the format of the body.
- Accept tells the server what response formats the client can handle.
- Authorization carries credentials or tokens.
- User-Agent can identify the calling application.
- Trace or request IDs help debugging across services.

QUICK REVISION NOTES

- A URL has scheme, host, path, and optional query string.
- GET reads; POST creates; PUT replaces; PATCH partially updates; DELETE removes.
- Headers carry metadata.
- Bodies carry structured request data.
- Query parameters commonly filter, sort, search, or paginate.

Practice Questions**Q1** What is the path in an API URL?**Q2** When would PATCH be preferable to PUT?**Q3** What is Content-Type used for?**Q4** What is a query parameter?**Hands-on Exercises****1** Write URLs for filtering applications by status.

2 Write a POST request body for creating a contact.

03

REST and Resource-Oriented Design

REST is a popular way to design web APIs around resources such as users, jobs, interviews, and certificates. The URL identifies the resource; the HTTP method describes the action.

LEARNING OBJECTIVES

- Explain REST at a beginner level
- Recognize resources and collections
- Design readable endpoint paths
- Understand stateless requests
- Use nouns instead of action-heavy URLs
- Describe CRUD operations

Think in resources

Need	Resource-style endpoint
List interviews	GET /interviews
Get one interview	GET /interviews/83
Create interview	POST /interviews
Update score	PATCH /interviews/83
Delete draft	DELETE /interviews/83

DESIGN HABIT

Prefer /applications/42 over /getApplicationById?id=42 when a resource-oriented design fits. The method already tells you that GET means read.

Collections and individual resources

/applications usually represents the collection. /applications/42 represents one member of that collection. Nested relationships can be useful, but deep nesting quickly becomes hard to maintain.

ENDPOINTS

GET /users/17/applications
GET /applications/42

Stateless does not mean no data

A stateless request contains enough information for the server to process that request without depending on hidden conversational state from a previous call. The server can still store users, orders, sessions, and application records.

COMMON MISTAKE

Treating REST as a rigid law. REST is a design style. Real APIs often use pragmatic exceptions. Consistency and clear contracts matter more than forcing every operation into an awkward pattern.

QUICK REVISION NOTES

- REST commonly models nouns as resources.
- Collections and single resources use predictable paths.
- Methods provide the action.
- CRUD maps naturally to create, read, update, delete.
- Stateless requests should be understandable independently.

Practice Questions

- Q1** What is a resource?
- Q2** How would you represent one certificate with ID 9?
- Q3** Why are nouns usually preferred in resource paths?
- Q4** What does stateless mean?

Hands-on Exercises

- 1 Design endpoints for a simple task manager.
- 2 Map CRUD operations to HTTP methods for a contacts API.

04

JSON and Data Contracts

Most modern APIs exchange structured data as JSON. A useful API contract defines not just field names, but also types, required fields, allowed values, and how missing data is represented.

LEARNING OBJECTIVES

- Read JSON objects and arrays
- Identify strings, numbers, booleans, null, objects, and arrays
- Create nested JSON
- Distinguish syntax from schema
- Understand required vs optional fields
- Recognize breaking contract changes

JSON basics

JSON

```
{
  "id": 42,
  "company": "Acme",
  "applied": true,
  "salary": null,
  "skills": ["SQL", "Power BI"]
}
```

JSON value	Example
String	"Data Analyst"
Number	42
Boolean	true
Null	null

Array	["SQL","Python"]
Object	{"city":"Pune"}

Contract thinking

If an API says score is a number from 0 to 100, sending "excellent" is syntactically valid JSON but contractually wrong. The contract gives meaning beyond syntax.

PROFESSIONAL HABIT

Document field type, required/optional status, allowed values, examples, and error behavior. Good API documentation prevents entire classes of integration bugs.

A response with nested data

JSON RESPONSE

```
{
  "candidate": {"id": 17, "name": "Mira"},
  "readiness": {"score": 78, "level": "job-ready"},
  "nextActions": ["Mock interview", "Tailor resume"]
}
```

COMMON MISTAKE

Assuming missing and null always mean the same thing. Some contracts use null to mean “known to have no value” while an omitted field may mean “not requested,” “not available,” or “not part of this response.”

QUICK REVISION NOTES

- JSON uses objects and arrays to structure data.
- Property names are strings.
- Values can be strings, numbers, booleans, null, arrays, or objects.
- A schema or contract defines meaning beyond valid JSON syntax.
- Changing field names or types can break clients.

Practice Questions

Q1 Name six JSON value types.

Q2 What is the difference between JSON syntax and an API contract?

Q3 Why can changing a field type be breaking?

Q4 What can null mean?

Hands-on Exercises

- 1 Write JSON for a candidate profile.
- 2 Create JSON containing a nested address and an array of skills.

05

Status Codes, Errors, and Debugging

A response is more than its body. The status code tells you the broad outcome, headers add context, and a structured error body explains what went wrong.

LEARNING OBJECTIVES

- Interpret common 2xx, 4xx, and 5xx status codes
- Distinguish client and server errors
- Read structured error responses
- Use request IDs
- Debug one layer at a time
- Avoid retrying every failure blindly

Status-code families

Range	Meaning	Examples
2xx	Success	200 OK, 201 Created, 204 No Content
4xx	Client/request problem	400, 401, 403, 404, 409, 422, 429
5xx	Server-side failure	500, 502, 503, 504

Important beginner codes

Code	Common meaning
200	Request succeeded
201	Resource created
204	Succeeded, no response body
400	Bad request

401	Authentication required/invalid
403	Authenticated but not allowed
404	Resource not found
409	Conflict with current state
422	Validation failed
429	Too many requests
500	Unexpected server error

ERROR RESPONSE

HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

```
{"error": "validation_failed", "field": "email", "message": "Enter a valid email"}
```

DEBUGGING CHECKLIST

Check URL -> method -> query parameters -> headers -> authentication -> body syntax -> field values -> status code -> error body -> server logs/request ID if available.

Retry with judgment

A 503 may be temporary and retryable. A 422 usually means the request must change first. A 401 may require refreshing credentials. Good clients respond differently to different classes of failure.

QUICK REVISION NOTES

- 2xx means success.
- 4xx usually means the request or permissions need attention.
- 5xx means the server or upstream dependency failed.
- Error bodies should be machine-readable and human-useful.
- Debug systematically instead of changing many things at once.

Practice Questions

Q1 What is the difference between 401 and 403?

Q2 When is 201 appropriate?

Q3 Why should a client not blindly retry a 422?

Q4 What is a request ID useful for?

Hands-on Exercises

1 Create three example error bodies.

2 Given five failure scenarios, choose a likely status code for each.

06

Authentication and API Security Basics

APIs often expose valuable or private data. Authentication answers “who are you?” while authorization answers “what are you allowed to do?”

LEARNING OBJECTIVES

- Distinguish authentication and authorization
- Understand API keys and bearer tokens
- Recognize OAuth at a high level
- Use HTTPS
- Protect secrets
- Understand least privilege and token expiry

Authentication vs authorization

Concept	Question	Example
Authentication	Who are you?	Validate a bearer token
Authorization	What may you do?	Can this user edit this application?

Common credential patterns

- API key: a secret identifier often used for server-to-server or developer access.
- Bearer token: sent in the Authorization header and accepted as proof of access.
- OAuth: a framework for delegated access, such as allowing an app to access a user account without learning the user password.
- Session cookie: common in browser applications, though not exclusive to APIs.

AUTHENTICATED REQUEST

```
GET /v1/profile
Authorization: Bearer eyJ...
Accept: application/json
```

SECURITY RULE

Never place real API keys or tokens in public repositories, screenshots, client-side code that exposes them, or shared documents. Use environment variables or a secret-management system.

Least privilege and expiry

A token should ideally have only the permissions it needs and should expire. Short-lived credentials reduce the damage if a token is leaked. Sensitive actions may require stronger checks than read-only operations.

QUICK REVISION NOTES

- Authentication identifies the caller.
- Authorization controls permitted actions.
- API keys and bearer tokens are credentials.
- OAuth supports delegated access.
- HTTPS protects data in transit.
- Secrets must not be committed to source control.
- Least privilege reduces risk.

Practice Questions

- Q1** What is authentication?
- Q2** What is authorization?
- Q3** Where is a bearer token commonly sent?
- Q4** Why should tokens expire?

Hands-on Exercises

- 1 Write a safe example Authorization header using a fake token.
- 2 List three places where real secrets should never be stored.

07

Testing APIs with curl, Postman, and Python

The fastest way to understand an API is to call it. Start with a simple GET, inspect everything, then move toward authenticated requests, bodies, and small client programs.

LEARNING OBJECTIVES

- Call APIs from curl
- Use a visual API client
- Send JSON bodies
- Inspect headers and timing
- Call an API from Python
- Handle errors explicitly

curl: a request you can save and share

TERMINAL

```
curl -i "https://api.example.com/v1/jobs?limit=5"
```

TERMINAL

```
curl -X POST "https://api.example.com/v1/applications" \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer DEMO_TOKEN" \  
-d '{"company":"Acme","role":"Analyst"}'
```

Visual API clients

Tools such as Postman-like clients make it easy to organize requests into collections, switch environments, inspect responses, and experiment with headers or bodies. They are especially useful while learning an unfamiliar API.

Python client example

PYTHON

```
import requests
```

```
response = requests.get(
    "https://api.example.com/v1/jobs",
    params={"limit": 5},
    timeout=10,
)

response.raise_for_status()
data = response.json()
print(data)
```

CLIENT HABIT

Always use timeouts in production code. Network calls can hang or fail. Treat an API call as an operation that can succeed, fail temporarily, fail permanently, or return unexpected data.

A simple debugging loop

- Reproduce the request in curl or a visual client.
- Confirm the exact URL and method.
- Inspect outgoing headers and body.
- Read the complete response status, headers, and body.
- Reduce the request to the smallest failing example.
- Add one complexity back at a time.

QUICK REVISION NOTES

- curl is excellent for reproducible HTTP requests.
- Visual clients help organize and inspect API calls.
- Python requests can turn API calls into programs.
- Use timeout and error handling.
- Debug with small reproducible requests.

Practice Questions

- Q1** Why is curl useful for debugging?
- Q2** What does `raise_for_status()` do conceptually?
- Q3** Why use a timeout?
- Q4** What information should you inspect in a response?

Hands-on Exercises

- 1 Call a public demo GET endpoint using curl.
- 2 Repeat the request in a visual client.
- 3 Write a Python script that prints one field from a JSON response.

08

CareerSense Mini Project: Build a Job Search API Client

The final project combines URLs, methods, JSON, status codes, authentication, and Python into a small job-search client. The API is fictional so you can focus on the integration pattern.

LEARNING OBJECTIVES

- Design an API interaction from a use case
- Create and list job applications
- Filter by status
- Handle errors
- Use a bearer token safely
- Summarize API data in Python

Scenario

Imagine CareerSense exposes a Job Tracker API. You want a tiny command-line client that can create applications, list them, and calculate how many have reached interview stage.

The fictional contract

Operation	Request
List applications	GET /v1/applications
Filter status	GET /v1/applications?status=interview
Create application	POST /v1/applications
Update status	PATCH /v1/applications/{id}
Get one record	GET /v1/applications/{id}

Create a record

HTTP REQUEST

```
POST /v1/applications
Authorization: Bearer DEMO_TOKEN
Content-Type: application/json
```

```
{
  "company": "Northstar",
  "role": "Data Analyst",
  "source": "Referral",
  "status": "applied"
}
```

RESULT

```
201 Created
{"id":104,"company":"Northstar","role":"Data Analyst","status":"applied"}
```

Build the Python client

PYTHON

```
import os
import requests

BASE_URL = "https://api.careersense.example/v1"
TOKEN = os.environ["CAREERSENSE_TOKEN"]
HEADERS = {"Authorization": f"Bearer {TOKEN}"}
```

```
def list_applications(status=None):
    params = {"status": status} if status else {}
    response = requests.get(
        f'{BASE_URL}/applications',
        headers=HEADERS,
        params=params,
        timeout=10,
    )
    response.raise_for_status()
    return response.json()
```

```
interviews = list_applications("interview")
print("Interview-stage applications:", len(interviews))
```

CAREER CONNECTION

This is the same integration pattern used in real applications: authenticate, call a resource endpoint, validate the outcome, parse JSON, and turn the response into a useful product behavior.

Extension challenges

- Add `create_application(company, role, source)`.
- Handle 401, 429, and 500 differently.
- Add pagination support.
- Export applications to CSV.
- Calculate application-to-interview conversion rate.
- Design a small UI that uses the same client functions.

QUICK REVISION NOTES

- Start from the API contract.
- Keep authentication outside source code.
- Use methods that match intent.
- Check status before trusting the body.
- Parse JSON into useful application behavior.
- Build integrations in small testable steps.

Practice Questions

- Q1** Which method creates an application?
- Q2** Where should the token come from in the Python example?
- Q3** How is status filtering represented?
- Q4** Why call `raise_for_status` before using the JSON?

Hands-on Exercises

- 1 Implement `create_application` using the fictional contract.
- 2 Add a function that updates an application to interview status.
- 3 Calculate the percentage of applications at interview stage.

Course Summary

You began with the simplest API idea: one program asking another program for a defined capability. From there you learned how HTTP describes the destination and action, how REST organizes resources, how JSON carries structured data, how status codes communicate outcomes, how authentication protects access, and how tools such as curl and Python turn the contract into real calls.

The most important habit is to read an API interaction as a complete system. Do not look only at the URL or only at the JSON. Read method, path, query, headers, authentication, body, status, response headers, and response body together. That is how integrations become understandable instead of mysterious.

PROFESSIONAL TAKEAWAY

APIs reward precision. Small details - one header, one field type, one status code, one missing permission - can decide whether an integration works. Build the habit of inspecting state systematically and documenting contracts clearly.

API Quick Revision Sheet

Topic	Remember
API	Contract for software communication
Client / Server	Client requests; server processes and responds
HTTP methods	GET read, POST create, PUT replace, PATCH partial update, DELETE remove
Headers	Metadata such as Content-Type and Authorization
Query parameters	Filters, search, sort, pagination options
JSON	Structured data format
2xx	Success
4xx	Request/auth/permission/client-side issue
5xx	Server/upstream failure
401 vs 403	Authentication problem vs not permitted
REST	Resource-oriented HTTP design style
Bearer token	Credential commonly sent in Authorization header
HTTPS	Encrypts transport
curl	Reproducible command-line HTTP client

Glossary

Term	Plain-English meaning
API	A defined interface that allows software systems to communicate.
Endpoint	A specific API address and capability.
Client	The caller making the request.
Server	The system receiving and processing the request.
HTTP	The protocol commonly used by web APIs.
Method	The requested action, such as GET or POST.
Header	Metadata attached to a request or response.
Body	The payload carried by a request or response.
Query parameter	A key-value option added to a URL.
JSON	A text format for structured data.
REST	A resource-oriented style for designing HTTP APIs.
Resource	A domain object exposed through an API.
Status code	Numeric summary of the HTTP outcome.
Authentication	Proving who the caller is.
Authorization	Checking what the caller may do.
API key	A secret credential used by some APIs.
Bearer token	A token presented as proof of access.
OAuth	A framework for delegated authorization.
Rate limit	A restriction on how frequently an API may be called.
Pagination	Splitting large result sets into manageable pages.

Schema	Rules describing the structure and types of data.
Idempotent	An operation that can be repeated without additional effect beyond the first successful application.
Timeout	Maximum time a client waits for an operation.
Webhook	A server-to-server callback sent when an event happens.

20-Question Practice Quiz

1. What is the main purpose of an API?
2. What are the roles of client and server?
3. Which HTTP method is normally used to read data?
4. Which method is commonly used to create a resource?
5. What is the difference between a path and a query parameter?
6. What does Content-Type describe?
7. What is a REST resource?
8. Name three JSON value types.
9. What does 201 Created communicate?
10. What broad category does 404 belong to?
11. What is the difference between 401 and 403?
12. Why should clients treat 429 specially?
13. What is authentication?
14. What is authorization?
15. Where is a bearer token commonly sent?
16. Why must real API secrets stay out of source control?
17. What does HTTPS protect?
18. Why is curl useful?
19. Why should network clients use timeouts?
20. What are the major pieces you should inspect when debugging a failed API request?

Answer Key

1. It provides a defined contract for one software system to communicate with another.
2. The client sends the request; the server processes it and sends a response.
3. GET.
4. POST.
5. The path identifies the resource/capability; query parameters add options such as filters or pagination.
6. The media format of the body, for example application/json.
7. A domain object or collection exposed through predictable URLs.
8. Examples: string, number, boolean, null, array, object.
9. The request succeeded and a new resource was created.
10. 4xx client/request category.
11. 401 indicates missing/invalid authentication; 403 means the caller is known but not allowed.
12. It means the caller has exceeded a rate limit and should generally slow down or wait according to the contract.
13. Proving the identity of the caller.
14. Checking whether the caller is permitted to perform an action.
15. In the Authorization header, commonly as Authorization: Bearer <token>.
16. Secrets can be copied, leaked, or abused if committed or shared publicly.
17. Data in transit between endpoints.
18. It makes HTTP requests reproducible, inspectable, and easy to share.
19. Network calls can hang or take too long; a timeout keeps the client from waiting forever.
20. URL, method, query parameters, headers, authentication, body, status code, response headers, response body, and request/server logs when available.

Keep Building

You now have the mental model required to explore real APIs with confidence. Choose one API that matters to your work, read its authentication section, make one successful GET request, then build outward slowly. Professional API skills are built by repeating that cycle: read the contract, make a precise request, inspect the response, and turn the result into useful software.

NEXT STEPS

Explore APIs for AI tools, payments, GitHub, cloud services, maps, communications, analytics, and the products you already use. The vocabulary in this handbook transfers across all of them.

More CareerSense learning resources
careersenseai.com/resources